

Aseguramiento de la Calidad en Plataformas de Comercio Electrónico: Un Caso de Estudio de Pruebas de Software sobre OpenCart.

Aguero Guerreros Luis Daniel, Quispe Bejar Garlet Analy, Suasaca Pacompia Alvaro Gustavo, Valdiviezo Tovar Alexander, Yucra Machacca Mariana Grethel

Escuela Profesional de Ingeniería de Sistemas

Universidad Nacional de San Agustín

laguero@unsa.edu.pe, gquispeb@unsa.edu.pe, asuasaca@unsa.edu.pe, avaldiviezot@unsa.edu.pe, myucrama@unsa.edu.pe

Abstract— Actualmente, el aseguramiento de la calidad del software es un factor crítico en el desarrollo de plataformas de comercio electrónico, donde fallas no detectadas generan pérdidas económicas y de confianza del usuario. Este artículo presenta el diseño, implementación y ejecución de un proceso de pruebas de software sobre OpenCart, un sistema de comercio electrónico de código abierto ampliamente adoptado. Se aplicó una metodología en tres niveles -pruebas unitarias, de integración y de sistema/aceptación- sobre seis componentes críticos: inventario, autenticación, carrito de compras, checkout, catálogo y reseñas, empleando PHPUnit, Composer y GitHub Actions dentro de un flujo de integración continua. Los resultados evidencian una cobertura promedio de líneas de 93.63% y de métodos de 84.37% en los módulos evaluados, con un índice CRAP promedio de 37.09, lo que refleja una calidad de código mayormente aceptable y áreas de riesgo moderado en autenticación y reseñas. El estudio aporta evidencia práctica sobre la aplicación de pruebas automatizadas en sistemas e-commerce reales y sienta una base metodológica reutilizable para futuros trabajos de aseguramiento de calidad en plataformas similares.

Keywords— pruebas de software, aseguramiento de calidad, pruebas unitarias, PHPUnit, comercio electrónico, integración continua, cobertura de código, OpenCart.

I. INTRODUCCIÓN

El presente trabajo de investigación busca analizar y documentar el proceso de diseño, implementación y ejecución de pruebas unitarias sobre un sistema de comercio electrónico basado en OpenCart. El proyecto se desarrolló con el propósito de fortalecer la calidad del software mediante la validación temprana de funcionalidades clave, reduciendo la probabilidad de errores en etapas posteriores del desarrollo. La evaluación se centró en componentes críticos del sistema, como autenticación, carrito de compras, inventario y procesos de negocio relacionados con la experiencia del usuario.

La importancia de este estudio radica en que las pruebas unitarias permiten verificar el comportamiento de cada módulo de forma aislada, lo que facilita la detección de fallas

y mejora la mantenibilidad del código. En un entorno de comercio electrónico, donde la disponibilidad y el correcto funcionamiento de los procesos de compra son esenciales, este tipo de pruebas se convierte en una herramienta estratégica para asegurar la estabilidad del sistema. Asimismo, el proyecto aporta una base sólida para futuras ampliaciones, integraciones y automatización de validaciones.

Durante el desarrollo del trabajo se incorporaron herramientas como PHP, PHPUnit y GitHub, con el objetivo de estructurar un flujo de trabajo ordenado y reproducible. La documentación generada, junto con la implementación de pruebas, permitió consolidar una propuesta metodológica útil para el aprendizaje y la aplicación de buenas prácticas en pruebas de software. Este artículo presenta el caso de estudio, la metodología aplicada y los resultados obtenidos durante la ejecución del proyecto.

II. ANTECEDENTES

1. Panorama general: técnicas de prueba, QA en e-commerce, cobertura, CI

La calidad del software se consolida como una etapa crítica del ciclo de vida de desarrollo, orientada a detectar fallas, errores y desviaciones respecto de los requisitos antes de que impacten al usuario final [1]. Jamil et al. Clasifican las estrategias de prueba en unitarias, de integración, de sistema y de aceptación, y destacan que cada nivel aporta un tipo distinto de evidencia sobre la corrección del software, por lo que su combinación resulta mas efectiva que la aplicación aislada de una sola técnica [6].

En el ámbito específico del comercio electrónico, Bhanushali documenta un caso de una tienda en línea que, tras incorporar pruebas de regresión automatizadas, monitoreo de rendimiento y pruebas centradas en la experiencia del usuario, redujo de forma sustancial sus fallas en producción y mejoró sus tiempos de carga, evidenciando que la inversión en aseguramiento de calidad tiene un retorno medible en plataformas transaccionales [7].

Un debate relevante en la literatura es la validez de la cobertura de código como indicador único de calidad de una suite de pruebas. Inozemtseva y Holmes muestran, mediante un estudio empírico sobre sistemas Java de gran escala, que la cobertura no está fuertemente correlacionada con la capacidad

real de una suite para detectar defectos, por lo que recomiendan complementar con otras métricas antes de usarla como criterio de aceptación [8]. Esta observación motiva que, en el presente trabajo, la cobertura se interprete siempre junto con el índice CRAP y el número de defectos detectados, y no de forma aislada.

La integración continua (CI) también ha sido objeto de amplia investigación empírica. Soares et al., en una revisión sistemática de 101 estudios, concluyen que la CI aporta beneficios consistentes en productividad, detección temprana de errores y confianza del equipo de desarrollo, aunque también introduce retos técnicos y de proceso relacionados con la configuración y el mantenimiento de los pipelines [9]. Estos hallazgos respaldan la decisión metodológica de este proyecto de incorporar GitHub Actions como eje del flujo de trabajo.

Otra línea de investigación activa es la automatización del oráculo de prueba, es decir, el mecanismo que determina si el resultado observado de una prueba es correcto. Cheon y Good analizan cómo la calidad de las aserciones (assertions) determina la capacidad de una prueba unitaria para revelar fallas, y proponen lineamientos para escribir aserciones mas efectivas en frameworks como PHPUnit y JUnit [10].

De la revisión anterior se observa que, si bien existe abundante literatura sobre técnicas de prueba, métricas de cobertura e integración continua de manera general, son escasos los estudios empíricos que apliquen estos conceptos de forma conjunta sobre un sistema PHP de comercio electrónico real y de código abierto como OpenCart. Este trabajo busca contribuir a cerrar dicha brecha mediante un caso de estudio reproducible.

2. Oráculos de prueba, aserciones, índice CRAP

Un reto central en las pruebas automatizadas es el llamado problema del oráculo de prueba: definir el mecanismo que permite decidir si el resultado observado de una ejecución es correcto o no. Para abordarlo, se han propuesto oráculos basados en aserciones, especificaciones formales y modelos abstractos del comportamiento esperado del sistema.

Complementando la noción de oráculo, la calidad interna del código bajo prueba también puede cuantificarse mediante métricas que combinan complejidad y cobertura. El índice CRAP (Change Risk Anti-Patterns), propuesto por Savoia y Evans, integra la complejidad ciclomática de un método con su porcentaje de cobertura para estimar el riesgo de que ese método introduzca defectos al ser modificado [11]. Un método complejo y bien cubierto por pruebas obtiene un puntaje bajo, mientras que un método complejo y no probado obtiene un puntaje desproporcionadamente alto.

En este trabajo se adoptan ambas nociones de forma práctica: las aserciones de PHPUnit se diseñaron siguiendo el principio de verificar comportamiento observable y no detalles internos de implementación [10], mientras que el índice CRAP se utilizó como criterio adicional -junto con la cobertura de líneas y métodos- para priorizar qué módulos requieren más atención, tal como se detalla en la sección de Resultados.

III. CASO ESTUDIO

A continuación se describe el software evaluado, su contexto histórico y los seis componentes seleccionados para el estudio.

1. OpenCart: Contexto y Descripción

OpenCart es una plataforma de comercio electrónico de código abierto, distribuida bajo la licencia GNU General Public License v3 (GPL v3). Desarrollada originalmente por Daniel Kerr en 2009, OpenCart se ha posicionado como una solución robusta para emprendedores, pequeñas y medianas empresas que requieren una tienda en línea funcional y personalizable. La plataforma cuenta con una comunidad activa de desarrolladores y miles de instalaciones en todo el mundo, lo que la convierte en un objeto de estudio relevante para investigaciones en aseguramiento de calidad de software [3].

OpenCart está construida sobre una arquitectura modular basada en PHP, siendo compatible con PHP 8.0 o superior en sus versiones más recientes. La plataforma implementa un patrón MVC (Model-View-Controller) adaptado, separando claramente la lógica de negocio (catálogo, carrito, gestión de pedidos) de la presentación, lo que facilita tanto el desarrollo como la implementación de pruebas. El sistema de bases de datos soporta múltiples motores como MySQL, PostgreSQL y MariaDB.

2. Historia y Evolución Técnica

OpenCart ha evolucionado significativamente desde su lanzamiento inicial. Las versiones modernas (3.0 en adelante) incorporan mejoras importantes en seguridad, rendimiento y estructura de código. Versiones recientes han adoptado estándares PHP modernos como PSR-4 para autoloading, mejoras en validación de entrada, y énfasis en seguridad contra vulnerabilidades OWASP. La plataforma soporta extensiones y modificaciones extensas, permitiendo adaptación a requerimientos específicos de negocio.

3. Componentes Identificados para el estudio

Para este trabajo se identificaron y seleccionaron seis componentes críticos que representan las funcionalidades principales de cualquier plataforma de e-commerce moderna:

a. Módulo de Gestión de Inventario

Este componente es central para la operación del negocio. Gestiona:

- Stock de productos y variantes
- Validación de disponibilidad en catálogo, carrito y checkout
- Manejo de opciones con impacto en inventario
- Estados de disponibilidad y datas de disponibilidad futura
- Integración con API para consultas externas
- Cantidad mínima de compra y políticas de overstocking

b. Módulo de Autenticación

Encargado de la seguridad de acceso al sistema:

- Registro de nuevos usuarios
- Autenticación y validación de credenciales
- Protección contra ataques de fuerza bruta
- Gestión de tokens CSRF (Cross-Site Request Forgery)
- Hash seguro de contraseñas
- Recuperación de contraseña

c. Módulo de Carrito de Compras

Gestiona la experiencia de compra:

- Agregar, modificar y eliminar productos del carrito
- Cálculo automático de totales y descuentos
- Validación de stock antes de procesar
- Persistencia de carrito
- Aplicación de promociones y cupones
- Gestión de impuestos

d. Módulo de Checkout y Pago

Orquesta el proceso de finalización de compra:

- Validación de dirección de facturación y envío
- Selección de métodos de envío
- Selección de métodos de pago
- Confirmación de orden
- Generación de pedido

e. Módulo de Catálogo y Búsqueda

Facilita descubrimiento de productos:

- Listado de productos con paginación
- Búsqueda textual y filtrado avanzado
- Ordenamiento por precio, popularidad, fecha
- Gestión de categorías y subcategorías
- Visualización de precios y descuentos
- Disponibilidad de stock visible

f. Módulo de Reseñas de Productos

Implementa social proof y feedback de usuarios:

- Creación y validación de reseñas
- Calificación por estrellas
- Moderación de contenido
- Cálculo de promedio de ratings
- Ordenamiento y filtrado de reseñas
- Eliminación y reporte de contenido inapropiado

IV. METODOLOGÍA

1. Enfoque General y Marco de Trabajo

La metodología empleada en este proyecto combina principios de ingeniería de software moderna con prácticas de desarrollo ágil. Se adopta un enfoque en capas de pruebas, reconociendo que cada nivel agrega valor y detecta defectos diferentes [2]. La estructura de trabajo se organiza en tres hitos secuenciales que construyen complejidad progresivamente:

- Hito 1: Pruebas Unitarias con énfasis en cobertura individual de componentes

- Hito 2: Pruebas de Integración e Integración Continua con despliegue automatizado
- Hito 3: Pruebas de Sistema y Aceptación con pipeline completo CI/CD

2. Herramientas y Tecnologías Empleadas

a. Framework de Pruebas: PHPUnit

PHPUnit es el estándar de facto para pruebas unitarias en PHP. Implementa el framework xUnit, permitiendo estructurar pruebas de forma sistemática y automatizable. Características principales utilizadas [4]:

- Assertions: Verificaciones granulares de comportamiento esperado vs. actual
- Mocks y Stubs: Aislamiento de componentes para pruebas unitarias puras
- Fixtures: Datos iniciales reutilizables para tests
- Data Providers: Pruebas parametrizadas para múltiples escenarios
- Code Coverage: Análisis de cobertura con reportes HTML detallados

b. Gestor de Dependencias: Composer

Composer facilita la gestión de dependencias PHP de forma declarativa mediante `composer.json`. Se utiliza para:

- Instalación y versionado de PHPUnit
- Dependencias del framework OpenCart
- Autoloading PSR-4 para clases del proyecto
- Scripts personalizados para automatización

c. Control de Versiones y Colaboración

La plataforma GitHub fue empleada para múltiples funciones [5]:

- GitHub Issues: Tracking detallado de bugs, features y tareas.
- GitHub Projects: Tablero Kanban visual para seguimiento ágil.
- GitHub Wiki: Documentación centralizada
- GitHub Pages: Publicación de reportes de cobertura HTML y documentación en línea,
- GitHub Actions: Pipelines CI/CD automatizados

3. Proceso de Planificación y Análisis de Requisitos

Fase 1: Extracción de Requisitos Funcionales

Se realizó análisis detallado de cada módulo, extrayendo requisitos funcionales específicos. Para los módulos

- Stock de productos
- Variantes y opciones
- Validación en carrito/checkout
- Validación por API
- Estados de stock

Este análisis se documentó en archivos Markdown individuales en `docs/Requisitos-funcionales/`, permitiendo

trazabilidad completa entre requisitos, casos de prueba y resultados.

Fase 2: Diseño de Casos de Prueba

Para cada requisito funcional se diseñaron múltiples casos de prueba considerando:

- Camino feliz (happy path): Casos donde todo funciona correctamente
- Casos límite: Valores en bordes de validación
- Casos de error: Comportamiento ante entradas inválidas
- Casos de integración: Interacciones entre componentes

Fase 3: Especificación de Escenarios de Prueba

Cada caso de prueba específica:

- Precondiciones requeridas
- Pasos de ejecución detallados
- Resultado esperado observable
- Datos de entrada y salida
- Criterios de éxito/fallo

V. RESULTADOS

PRUEBAS UNITARIAS

1. Módulo: Gestión de Inventario

Resumen de Ejecución

Métrica	Valor
Tests implementados	54
Tests superados	54 (100 %)
Líneas de código cubiertas	100 % (123/123 LOC)
Métodos cubiertos	100 % (14/14)
Tiempo de ejecución	142 ms

2. Módulo: Login y Registro

Resumen de ejecución

Métrica	Valor
Tests implementados	46 de 114
Tests superados	46/46 (100 %)
Líneas de código cubiertas	89.04 % (130/146 LOC)
Métodos cubiertos	76.92 % (20/26)
Tiempo de ejecución	189 ms
Defectos detectados	6 (5 corregidos)

3. Módulo: Carrito de Compras

Resumen de ejecucion

Métrica	Valor
Tests implementados	34 de 60
Tests superados	34/34 (100 %)
Líneas de código cubiertas	94.44 % (85/90 LOC)
Métodos cubiertos	88.24 % (15/17)
Tiempo de ejecución	246 ms
Defectos detectados	1 (Corregido)

4. Módulo: Catalogo y Búsqueda

Resumen Planificado

Métrica	Valor
Casos de prueba diseñados	127
Casos implementados	0 (Fase 2)
Cobertura actual del código base	100 % (52/52 LOC)
Métodos del gestor cubiertos	100 % (20/20)

5. Módulo:Checkout y Pago

Resumen Planificado

Métrica	Valor
Casos de prueba diseñados	31
Casos implementados	0 (Fase 2)
Cobertura actual del código base	100 % (101/101 LOC)
Métodos del gestor cubiertos	100 % (19/19)

6. Módulo: Sistema de Reseñas

Resumen planificado

Métrica	Valor
Casos de prueba diseñados	65
Casos implementados	0 (Fase 2)
Cobertura actual del código base	86.30 % (63/73 LOC)
Métodos del gestor cubiertos	72.22 % (13/18)

Cobertura de Código por Módulo

La siguiente tabla resume los resultados obtenidos tras la ejecución de las pruebas unitarias automatizadas para cada módulo del sistema.

Módulo	Cobertura de Líneas	Cobertura de Métodos	Índice CRAP	Estado
InventoryManager	100 % (123/123)	100 % (14/14)	37.00	✓ Excelente
CatalogManager	100 % (52/52)	100 % (20/20)	28.00	✓ Excelente
CheckoutManager	100 % (101/101)	100 % (19/19)	41.00	✓ Excelente
CartManager	94.44 % (85/90)	88.24 % (15/17)	33.19	● Muy bueno
AuthenticationManager	89.04 % (130/146)	76.92 % (20/26)	67.06	● Bueno
ReviewManager	86.30 % (63/73)	72.22 % (13/18)	42.91	● Bueno
Promedio General	93.63 %	84.37 %	37.09	✓ Muy bueno

AuthenticationManager.php	89.04%	130 / 146	76.92%	20 / 26
CartManager.php	94.44%	85 / 90	88.24%	15 / 17
CatalogManager.php	100.00%	52 / 52	100.00%	20 / 20
CheckoutManager.php	100.00%	101 / 101	100.00%	19 / 19
InventoryManager.php	100.00%	123 / 123	100.00%	14 / 14
ReviewManager.php	86.30%	63 / 73	72.22%	13 / 18

PRUEBAS DE INTEGRACIÓN

Panorama general de las pruebas de integración

Las pruebas de integración tuvieron como propósito verificar la correcta comunicación entre los módulos de OpenCart y comprobar la consistencia de los datos intercambiados durante la ejecución de los principales procesos de negocio. Estas pruebas permitieron evaluar operaciones relacionadas con la autenticación de clientes, gestión del carrito, catálogo, inventario, checkout, pagos, pedidos, envíos, descuentos y reseñas.

Para su ejecución se utilizó un entorno automatizado mediante GitHub Actions, compuesto por Ubuntu 24.04, PHP 8.2.32, PHPUnit 10.5.64 y MySQL 8.0. El pipeline preparó la base de datos, instaló OpenCart, ejecutó las suites de integración y realizó pruebas de humo sobre el frontend y el panel administrativo, tanto en el entorno local como mediante un acceso público temporal.

Resumen de escenarios diseñados

Se diseñaron un total de 60 escenarios de pruebas de integración, orientados a comprobar la interacción entre los diferentes componentes del sistema. Los escenarios contemplaron flujos principales, validaciones de datos, reglas de negocio, manejo de errores, condiciones límite, persistencia

en la base de datos y procesos que involucran más de un módulo.

Del total de escenarios diseñados, 55 fueron implementados como pruebas automatizadas con PHPUnit, mientras que 5 quedaron identificados como pendientes de automatización. Los escenarios automatizados realizaron un total de 111 assertions y finalizaron sin errores ni fallos.

Los cinco escenarios pendientes corresponden a las siguientes validaciones:

- restauración del stock cuando falla la creación de una orden;
- conservación del carrito después de iniciar sesión;
- consistencia del precio entre búsqueda, detalle y carrito;
- actualización del promedio al aprobar una reseña;
- prevención de confirmaciones duplicadas de pago.

Los escenarios fueron priorizados según el impacto que una falla podría generar en la operación del sistema y en la experiencia del usuario. Las integraciones relacionadas con carrito, checkout, inventario, pagos y pedidos recibieron una criticidad mayor debido a su participación directa en el proceso de compra.

Matriz de integración crítica

La siguiente matriz presenta las integraciones evaluadas, su nivel de criticidad y la cantidad de escenarios diseñados. La clasificación considera la importancia del proceso, la dependencia entre los módulos y las consecuencias que podría producir una falla.

Módulo A	Módulo B	Nivel de criticidad	Escenarios diseñados
Carrito	Checkout	● Crítica	10
Checkout	Inventario	● Crítica	5
Pagos	Orden e historial	● Crítica	5
Pedidos	Detalle y totales	● Crítica	4
Clientes	Autenticación	● Alta	11
Catálogo	Búsqueda	● Alta	6
Productos	Inventario	● Alta	5
Envíos	Pedidos y totales	● Alta	4
Descuentos	Totales	● Alta	4
Reseñas	Catálogo	● Media	6
Total			60

Las integraciones clasificadas como críticas se relacionan directamente con la generación de órdenes, actualización del

inventario, confirmación de pagos y cálculo de los importes de una compra. Una falla en estos procesos podría ocasionar ventas sin disponibilidad, duplicidad de pagos, pedidos incompletos o inconsistencias en los totales.

Las integraciones de criticidad alta intervienen en la autenticación, búsqueda de productos, disponibilidad del inventario, cálculo del envío y aplicación de descuentos. Aunque algunas no interrumpen inmediatamente la creación de una orden, pueden afectar la continuidad del proceso de compra o generar información incorrecta.

La integración entre reseñas y catálogo fue clasificada con criticidad media porque sus fallos afectan principalmente la presentación de calificaciones y opiniones, pero no impiden directamente la realización de una compra.

Estado	Cantidad	Porcentaje
Automatizados	55	91.67%
Pendientes de automatización	5	8.33%
Total	60	100%

La documentación y trazabilidad de los 60 escenarios se almacenó en tests/integracion/escenarios/README.md. En este documento se relaciona cada escenario con su identificador, descripción, archivo PHPUnit, método automatizado y estado de implementación.

Resultados de la ejecución

Los resultados evidenciaron que las integraciones automatizadas funcionaron correctamente en el entorno evaluado. Asimismo, las pruebas de humo del frontend y del panel administrativo respondieron satisfactoriamente con el código HTTP 200.

PRUEBAS DE SISTEMA Y NO FUNCIONALES

Resumen general del Plan

La siguiente tabla resume los diferentes tipos de pruebas contemplados para la validación integral del sistema, indicando la cantidad de escenarios diseñados, su nivel de prioridad y la herramienta prevista para su ejecución.

Tipo de Prueba	Cantidad de Escenarios	Prioridad	Herramienta
Pruebas funcionales End-to-End (E2E)	7 flujos	Alta	Ejecución manual / Navegador
Pruebas de rendimiento (Carga)	4 escenarios	Alta	Manual / JMeter
Pruebas de estrés	6 escenarios (incluye Spike y Endurance)	Media	Manual / JMeter
Compatibilidad entre navegadores	4 navegadores	Alta	Chrome DevTools

Compatibilidad entre dispositivos	4 resoluciones	Alta	Chrome DevTools
Adaptabilidad Responsive	7 resoluciones	Media	Chrome DevTools
Pruebas de seguridad	7 escenarios	Alta	Ejecución manual
Pruebas de regresión	5 escenarios	Media	PHPUnit + Manual
Requisitos No Funcionales (RNF) documentados	28 requisitos	—	—

PRUEBAS DE ACEPTACIÓN

Resumen consolidado del estado de ejecución de las 29 historias de usuario. El detalle completo de cada escenario y su evidencia está en el "Registro de Ejecución" al final de cada documento de módulo

Módulo	 Cumple	 Riesgo / Parcial	 Pendiente	 No cumple
Login y Registro	2	1	1	1
Catálogo y Búsqueda	5	0	0	0
Carrito de Compras	4	0	1	0
Checkout y Pago	1	1	2	0
Gestión de Inventario	4	1	0	0
Sistema de Reseñas	2	0	2	0

VI. CONCLUSIONES

El proceso de aseguramiento de calidad aplicado a los seis componentes críticos de OpenCart permitió ejecutar 134 pruebas unitarias, todas con resultado satisfactorio. La suite alcanzó una cobertura promedio de 93.63 % en líneas de código y 84.37 % en métodos, lo que evidencia que la estrategia de pruebas por capas adoptada resulta viable para evaluar un sistema de comercio electrónico desarrollado en PHP.

Los módulos InventoryManager, CatalogManager y CheckoutManager alcanzaron una cobertura completa y presentaron índices CRAP inferiores a 41, ubicándose dentro de un nivel de riesgo aceptable. En contraste, AuthenticationManager y ReviewManager registraron los índices CRAP más elevados, con valores de 67.06 y 42.91, respectivamente. Estos resultados indican que ambos componentes concentran una mayor complejidad ciclomática en relación con su cobertura y, por consiguiente, presentan un riesgo superior ante futuras modificaciones.

En el nivel de integración se diseñaron 60 escenarios, de los cuales 55 fueron implementados y ejecutados automáticamente mediante PHPUnit. Las pruebas realizaron 111 assertions y finalizaron sin errores ni fallos. Los resultados permitieron comprobar la interacción entre carrito, checkout, inventario, autenticación, catálogo, reseñas, pagos, pedidos, envíos, totales y descuentos.

Los cinco escenarios pendientes de automatización corresponden a la restauración del stock cuando falla una orden, conservación del carrito después del inicio de sesión, consistencia de precios entre búsqueda, detalle y carrito, actualización del promedio al aprobar una reseña y prevención de confirmaciones duplicadas de pago.

El pipeline implementado con Composer, PHPUnit y GitHub Actions permitió establecer un proceso reproducible para instalar las dependencias, preparar MySQL, configurar OpenCart y ejecutar automáticamente las pruebas. Además, las pruebas de humo del frontend y del panel administrativo respondieron correctamente con el código HTTP 200.

La combinación de pruebas unitarias, pruebas de integración, cobertura de código y automatización continua proporcionó una visión más completa de la calidad del sistema. Sin embargo, una cobertura elevada no garantiza por sí sola la efectividad de la suite, por lo que debe complementarse con pruebas de mutación, seguridad, rendimiento y validación de los requisitos no funcionales [8].

En conjunto, los resultados demuestran que la incorporación temprana de pruebas automatizadas reduce el riesgo de fallos en procesos esenciales del comercio electrónico. La trazabilidad establecida entre requisitos, escenarios, métodos automatizados y resultados proporciona una base que facilita la auditoría, el mantenimiento y la ampliación futura del proyecto.

VII. RECOMENDACIONES Y TRABAJOS FUTUROS

Se recomienda implementar los cinco escenarios de integración pendientes para completar la automatización de los 60 escenarios diseñados. Después de su incorporación, deberá ejecutarse nuevamente el workflow y actualizarse la matriz de trazabilidad con la cantidad definitiva de pruebas y assertions.

Se recomienda priorizar la refactorización de AuthenticationManager y ReviewManager, debido a que presentan los índices CRAP más elevados. La reducción de su complejidad puede realizarse mediante la separación de responsabilidades, extracción de métodos, simplificación de estructuras condicionales y ampliación de la cobertura de sus caminos alternativos.

Debe incorporarse una herramienta de pruebas de mutación para medir la capacidad real de la suite para detectar modificaciones incorrectas. Esta evaluación permitirá complementar los porcentajes de cobertura, considerando que una línea ejecutada no implica necesariamente que su comportamiento haya sido validado correctamente [8].

Se recomienda ampliar el pipeline de GitHub Actions con controles de calidad adicionales, como análisis estático, validación de estilo, auditoría de dependencias y publicación automática de reportes. También resulta conveniente almacenar los resultados como artefactos para conservar evidencia histórica de cada ejecución.

Las pruebas de rendimiento deben mantenerse dentro del proceso automatizado mediante k6, incorporando umbrales para tiempo de respuesta, tasa de errores y usuarios concurrentes. Las pruebas deben incluir escenarios de carga normal, estrés, picos y resistencia [17].

Las pruebas de seguridad deben ampliarse tomando como referencia la guía OWASP Web Security Testing Guide y el Secure Software Development Framework de NIST. Se recomienda evaluar autenticación, manejo de sesiones, inyección SQL, XSS, CSRF, configuración del servidor y protección de datos sensibles [14], [15].

También se recomienda ejecutar pruebas de compatibilidad en diferentes navegadores, dispositivos y resoluciones, así como evaluaciones de usabilidad y accesibilidad. Estas verificaciones permitirían valorar atributos de calidad que no son cubiertos por las pruebas unitarias y de integración, siguiendo el modelo de calidad de ISO/IEC 25010 [16].

Finalmente, puede explorarse la generación asistida de casos, datos y oráculos de prueba mediante modelos de lenguaje. No obstante, los resultados generados deben ser revisados por el equipo de QA para evitar assertions triviales, casos duplicados o validaciones que no representen correctamente los requisitos del sistema [10].

REFERENCIAS

- [1] I. Sommerville, *Software Engineering*, 9th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [2] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA, USA: Addison-Wesley Professional, 2009.
- [3] OpenCart Foundation, "OpenCart official documentation," 2024. [Online]. Available: <https://github.com/opencart/opencart>
- [4] PHPUnit, "PHPUnit 10.5 manual," 2024. [Online]. Available: <https://phpunit.de/documentation.html>
- [5] GitHub, "GitHub Actions documentation," 2024. [Online]. Available: <https://docs.github.com/en/actions>
- [6] M. A. Jamil, M. Arif, N. S. Awang Abubakar, and A. Ahmad, "Software testing techniques: A literature review," in *Proc. 6th Int. Conf. Inf. Commun. Technol. Muslim World (ICT4M)*, Jakarta, Indonesia, 2016, pp. 177-182.
- [7] A. Bhanushali, "Ensuring software quality through effective quality assurance testing: Best practices and case studies," *Int. J. Adv. Eng. Res.*, vol. 26, no. 4, pp. 1-18, Oct. 2023.
- [8] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *Proc. 36th Int. Conf. Softw. Eng. (ICSE)*, Hyderabad, India, 2014, pp. 435-445.
- [9] E. Soares, G. Sizilio, J. Santos, D. Alencar da Costa, and U. Kulesza, "The effects of continuous integration on software development: A systematic literature review," *Empirical Softw. Eng.*, vol. 27, no. 3, art. 78, 2022.
- [10] Y. Cheon and B. Good, "Writing effective test oracle assertions," in *Proc. 13th Int. Conf. Softw. Eng. Res. Innov. (CONISOFT)*, 2025, pp. 243-251.
- [11] A. Savoia and B. Evans, "CRAP: Change risk anti-patterns," *Google Testing Blog*, 2007. [Online]. Available: <http://www.crap4j.org>
- [12] ISO/IEC/IEEE, *ISO/IEC/IEEE 29119-2:2021—Software and Systems Engineering: Software Testing—Part 2: Test Processes*. Geneva, Switzerland: International Organization for Standardization, 2021. [Online]. Available: [ISO 29119-2:2021](https://www.iso.org/standard/75441.html).
- [13] International Software Testing Qualifications Board, *Certified Tester Foundation Level Syllabus, Version 4.0.1. ISTQB*, 2024. [Online]. Available: [ISTQB CTFL Syllabus v4.0.1](https://www.istqb.org/certification/certified-tester/certified-tester-foundation-level).

- [14] OWASP Foundation, Web Security Testing Guide. OWASP, 2026. [Online]. Available: OWASP WSTG.
- [15] M. Souppaya, K. Scarfone, and D. Dodson, Secure Software Development Framework (SSDF), Version 1.1, NIST Special Publication 800-218. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2022. [Online]. Available: NIST SP 800-218.
- [16] ISO/IEC, ISO/IEC 25010:2023—Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model. Geneva, Switzerland: International Organization for Standardization, 2023. [Online]. Available: ISO/IEC 25010:2023.
- [17] Grafana Labs, “Grafana k6 documentation,” 2026. [Online]. Available: <https://grafana.com/docs/k6/latest/>.
- [18] Composer, “Composer documentation,” 2026. [Online]. Available: <https://getcomposer.org/doc/>.
- [19] ISO/IEC, ISO/IEC 25019:2023—Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE)—Quality-in-Use Model. Geneva, Switzerland: International Organization for Standardization, 2023. [Online]. Available: ISO/IEC 25019:2023.
- [20] ISO/IEC, ISO/IEC 25002:2024—Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE)—Quality Model Overview and Usage. Geneva, Switzerland: International Organization for Standardization, 2024. [Online]. Available: ISO/IEC 25002:2024.
- [21] Grafana Labs, “Automated performance testing,” Grafana k6 Documentation, 2026. [Online]. Available: <https://grafana.com/docs/k6/latest/testing-guides/automated-performance-testing/>.